

INTERNSHIP TRAINING REPORT

on

.NET Full Stack Development

Capgemini Technology Services India Limited

A Training Report

Submitted in partial fulfilment of the requirements for the award of degree of
Bachelor of Technology in Computer Science and Engineering

Submitted to

**LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB**



FROM 17/12/2025 TO 20/05/2026

Submitted By: Yelugotla Sanket | Reg. No.: 12223854

Submitted to: Dr. Harpreet Singh Bedi

Student Declaration

I, Yelugotla Sanket, Registration Number 12223854, hereby declare that the work done by me on the Internship Training Report on .NET Full Stack Development from December 2025 to May 2026, under the supervision of Ms. Tiwari Avantika, Sr. Manager, Capgemini Technology Services India Limited, and Dr. Harpreet Singh Bedi, Lovely Professional University, Phagwara, Punjab, is a record of original work carried out by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

I further declare that this report has not previously formed the basis for the award of any degree, diploma, associateship, fellowship, or any other similar title to any candidate at any university or institution.

The project described in this report — CapFinLoan, a microservices-based loan processing platform — was conceptualized, designed, developed, tested, and documented entirely by me during the internship period at Capgemini Technology Services India Limited. All third-party libraries and frameworks referenced in this report are duly credited.

Name of the Student: Yelugotla Sanket | 12223854

Signature of the Student: _____

Date: 06/06/2026

Acknowledgement

I would like to express my heartfelt gratitude to all those who supported me during this internship and guided me through the development of the CapFinLoan project.

First and foremost, I am deeply thankful to Ms. Tiwari Avantika, Sr. Manager at Capgemini Technology Services India Limited, for her mentorship, continuous guidance, and encouragement throughout the internship. Her insights into real-world software engineering practices, microservices architecture, and professional development were invaluable.

I am equally grateful to Dr. Harpreet Singh Bedi, Faculty Guide at Lovely Professional University, for his academic supervision, timely feedback, and constructive advice. His expertise and support were instrumental in shaping both the technical and documentation aspects of this report.

I extend my sincere thanks to the entire Capgemini team for providing an enriching environment with access to cutting-edge technologies, development tools, and professional practices. The structured pre-onboarding program, combined with the hands-on project experience, has significantly enhanced my technical capabilities in full-stack development, cloud-native patterns, and distributed systems design.

Finally, I am deeply grateful to my family, friends, and colleagues for their unwavering moral support and encouragement throughout this journey.

Table of Contents

Student Declaration	2
Acknowledgement.....	3
Table of Contents.....	4
Chapter 1: About Capgemini Technology Services	6
1.1 Introduction.....	6
1.2 History and Evolution.....	6
1.3 Services Offered.....	7
1.4 Technologies and Career Opportunities	8
Chapter 2: Project Undertaken — CapFinLoan	9
2.1 Abstract	9
2.2 Project Objectives.....	9
2.3 Scope of the Project	10
Chapter 3: Brief Description of Work Done.....	12
3.1 Technology Stack	12
3.2 System Architecture	12
3.2.1 High-Level Architecture Diagram.....	13
3.2.2 Microservices Overview	13
3.3 RabbitMQ Messaging Architecture	14
3.4 Saga Choreography Design.....	15
3.5 Authentication and Security Architecture	16
3.5.1 OTP-Based Email Signup	16
3.5.2 Google OAuth 2.0	17
3.5.3 JWT and API Gateway Security	17
3.6 Loan Application Lifecycle	19
3.7 API Gateway and Routes	20
3.8 Frontend Architecture	22
3.8.1 Applicant Pages	22
3.8.2 Admin Pages.....	22
3.9 AI Chatbot Integration.....	23
3.10 Use Case Diagram	24
3.11 Database Design	24
3.11.1 Database Overview.....	24
3.11.2 Entity-Relationship Diagram.....	25
3.11.3 Key Schema Details.....	26
3.12 Clean Architecture	27

3.13 Unit Testing Strategy	27
3.14 Docker and Deployment	28
3.14.1 Container Architecture.....	28
3.14.2 Environment Configuration	29
3.14.3 Development URLs	29
3.15 Design Patterns Used.....	30
3.16 Reliability and Fault Tolerance.....	30
Chapter 4: Conclusion.....	32
Chapter 5: References	34
Books and Academic References.....	34
Online Documentation and Resources	34

Chapter 1: About Capgemini Technology Services

1.1 Introduction

Capgemini is a global leader in consulting, technology services, and digital transformation. Founded in 1967 by Serge Kampf in Grenoble, France under the name Sogeti, the company has grown into one of the world's largest IT services and consulting organizations. Over the decades, Capgemini has expanded through strategic acquisitions and organic growth to become a trusted partner for businesses across industries looking to harness the power of technology to improve operations, increase efficiency, and achieve their strategic goals.

The company operates in more than 50 countries and serves clients across diverse industries including banking, healthcare, manufacturing, telecommunications, automotive, retail, and energy. With a workforce of over 340,000 employees worldwide, Capgemini is recognized for its innovation, customer-centric approach, and deep expertise in emerging technologies. The company consistently ranks among the top global IT service providers and has received numerous industry accolades for its digital transformation capabilities, sustainability commitments, and inclusive work culture.

In the Indian context, Capgemini has established a significant operational presence with major delivery centers in Bengaluru, Hyderabad, Pune, Mumbai, Chennai, Kolkata, and Noida. India serves as one of Capgemini's largest and most strategically important hubs, contributing significantly to software development, testing, cloud computing, cybersecurity, data analytics, and engineering services delivered to clients worldwide.

1.2 History and Evolution

Capgemini's journey began in 1967 when Serge Kampf founded Sogeti in Grenoble, France, as a modest data processing company. The name 'Capgemini' emerged in 1975 following the merger of Sogeti with CAP (Centre d'Analyse et de Programmation) and Gemini Computer Systems, creating a more formidable entity in the European IT landscape.

The 1990s and 2000s were transformative decades for the company. A landmark milestone came in 2000 with the acquisition of Ernst & Young Consulting, which significantly strengthened Capgemini's management consulting capabilities and gave the company a truly global reach. This acquisition nearly doubled the company's workforce and positioned Capgemini as a serious competitor to the world's leading consulting firms.

In 2020, Capgemini completed the acquisition of Altran Technologies, a world leader in engineering and R&D services. This strategic move transformed Capgemini into a global leader in engineering and product development, enabling the company to offer end-to-end innovation services from ideation through product design, embedded systems, semiconductor solutions, and

industrial automation. Today, Capgemini continues to invest heavily in digital transformation, cloud computing, artificial intelligence, and cybersecurity solutions, positioning itself at the forefront of the rapidly evolving technology landscape.

1.3 Services Offered

Capgemini provides a comprehensive portfolio of services to organizations worldwide. Its service offerings are structured around several core competencies that address the full spectrum of business and technology needs:

Consulting Services

Capgemini's consulting division helps organizations identify business challenges and develop effective strategies for transformation. The team assists clients in improving business processes, reducing operational costs, and implementing large-scale digital transformation initiatives. Consultants work across industries to design operating models, develop technology roadmaps, and guide organizational change.

Cloud Services

The company offers comprehensive cloud migration, cloud infrastructure management, and cloud-native application development services. Capgemini maintains strategic partnerships with all major cloud providers including Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP). Cloud advisory, migration, optimization, and managed services form a significant and growing part of the company's revenue.

Artificial Intelligence and Data Analytics

Capgemini leverages AI and advanced analytics to help businesses gain actionable insights from their data, automate routine operations, and improve decision-making processes. The company's AI capabilities span natural language processing, computer vision, predictive analytics, machine learning model development, and MLOps for enterprise-scale AI deployment.

Cybersecurity

Cybersecurity services include threat detection, risk management, security operations center (SOC) services, and compliance solutions. These services help organizations protect their digital assets, intellectual property, and sensitive customer information in an increasingly complex threat environment.

Application Development and Maintenance

Capgemini develops, tests, deploys, and maintains software applications for clients across different industries. The company employs modern software engineering practices including Agile and DevOps methodologies, continuous integration and delivery (CI/CD) pipelines, and microservices architecture to deliver high-quality software products.

Engineering and R&D Services

Through its engineering services division (formerly Altran), Capgemini provides solutions related to product design, embedded systems, semiconductor technologies, and industrial automation. This division serves clients in automotive, aerospace, industrial, and consumer electronics sectors.

1.4 Technologies and Career Opportunities

Capgemini works with a wide range of modern technologies to deliver innovative solutions. The primary technology areas include Artificial Intelligence (AI) and Machine Learning (ML), Cloud Computing platforms and DevOps automation, Cybersecurity frameworks and solutions, Big Data Analytics and Business Intelligence, Internet of Things (IoT), Blockchain, Robotic Process Automation (RPA), Digital Engineering, and Full-Stack Web and Mobile Development.

For fresh graduates, Capgemini offers structured career pathways across several roles. Common entry-level and early-career positions include Software Engineer, Test Engineer, Data Analyst, Cloud Engineer, Cybersecurity Analyst, Business Analyst, DevOps Engineer, and AI/ML Engineer. The company places great emphasis on continuous learning through technical certifications, leadership development programs, and industry-recognized training pathways.

Chapter 2: Project Undertaken — CapFinLoan

2.1 Abstract

CapFinLoan is a production-grade, microservices-based Loan Processing Platform developed during the internship at Capgemini Technology Services India Limited. The platform supports the complete digital lending journey — from user onboarding and loan application creation to document upload, administrative review, final decision-making, and email notifications — across a modern, distributed architecture designed to mirror real-world enterprise financial systems.

The backend is implemented as six ASP.NET Core 8 microservices plus one separate Python ChatbotService, each independently deployable and responsible for a distinct domain concern. These services are coordinated through an Ocelot API Gateway that handles JWT validation and route forwarding. Service-to-service communication is achieved through both synchronous HTTP calls and asynchronous RabbitMQ messaging using the MassTransit library, enabling robust, decoupled event-driven workflows.

The frontend is an Angular 18 single-page application built with Vite and styled using Tailwind CSS. It provides a seamless user experience for both loan applicants and administrators, featuring a multi-step loan application wizard, real-time status tracking, document management, an AI-powered chatbot assistant, and comprehensive admin reporting with data visualization. Authentication supports JWT-based sessions, Google OAuth 2.0, and OTP-based email verification.

The system follows Clean Architecture principles within each service, uses SQL Server 2022 with Entity Framework Core Code-First migrations with a database-per-service model, and is fully containerized using Docker and Docker Compose for reproducible local deployment. A unit testing suite built on NUnit and Moq covers all critical business logic across four services.

2.2 Project Objectives

The CapFinLoan project was conceived with a set of well-defined objectives that guided its design and implementation throughout the internship:

- Design and implement a full-stack Loan Processing Platform using Angular 18 (Vite + Tailwind CSS) on the frontend and ASP.NET Core 8 microservices on the backend, demonstrating end-to-end ownership of a complex software system.
 - Apply microservices architecture principles including database-per-service, API gateway routing, service discovery, and clean separation of domain concerns across seven independently deployable services (6 .NET backend services + 1 Python ChatbotService).
 - Implement asynchronous service communication using RabbitMQ and MassTransit for critical event flows including OTP delivery, application status synchronization, and decision notifications.
-

- Design and implement a choreography-based saga pattern that orchestrates the loan lifecycle across multiple services without a central orchestrator, enabling human-in-the-loop decision points for administrative review.
- Build a secure authentication system supporting multiple identity providers — email/password with PBKDF2 hashing, Google OAuth 2.0, and OTP-based email verification with JWT-based stateless sessions.
- Develop a complete admin dashboard with decision management, document verification workflows, and visual reporting with CSV export capabilities.
- Integrate an AI-powered chatbot using a locally hosted Groq API instance (llama-3.1-8b-instant model) with intent detection for loan application queries and voice input support via the Web Speech API.
- Containerize the entire platform using Docker and Docker Compose, enabling one-command full-stack deployment for both development and demonstration environments.
- Achieve meaningful test coverage using NUnit and Moq unit tests across the four core business services, validating all critical state transitions and business rule enforcement.
- Document the system comprehensively through HLD, LLD, API references, environment setup guides, and this internship report.

2.3 Scope of the Project

The scope of CapFinLoan encompasses the complete digital loan application and review lifecycle, spanning frontend, backend, messaging, security, and operations layers of a modern distributed system.

In-Scope Capabilities

- User registration via two-step OTP email verification flow and Google OAuth 2.0 single sign-on.
- Multi-step loan application wizard (4 steps: Personal Info, Employment Details, Loan Details, Review) with draft save and submit functionality.
- Document upload for four document types (ID Proof, Address Proof, Bank Statement, Income Proof) with per-document verification and rejection by administrators.
- Loan application lifecycle management with status transitions: Draft → Submitted → DocsPending → DocsVerified → UnderReview → Approved/Rejected.
- Admin dashboard with KPI cards, application queue management, document review interface, decision panel with sanction amount and interest rate configuration.
- Email notifications for OTP codes, approval decisions, and rejection notices via SMTP integration through NotificationService.
- AI chatbot assistant with intent detection for loan application queries and voice input support.
- Admin reports with bar charts, pie charts (using Recharts), and CSV export of application data.
- Full Docker Compose containerization with health checks and environment-driven configuration.

Out-of-Scope

- Live credit bureau integration — credit scoring is not implemented; loan decisions are made manually by administrators.

- Production payment processing or EMI calculation engine.
- Multi-tenancy or white-label bank branding.
- Kubernetes/Helm cloud deployment — recommended in the roadmap but not in current scope.

Chapter 3: Brief Description of Work Done

3.1 Technology Stack

CapFinLoan employs a carefully selected technology stack that reflects modern enterprise software engineering practices. Each technology was chosen for its production maturity, community support, and suitability for the specific role it fulfills within the system.

Layer	Technology	Purpose
Frontend	Angular21 + Tailwind CSS	SPA with routing, guards, Axios interceptors, applicant and admin workflows
Backend	ASP.NET Core 8 (.NET 8)	REST microservices, domain logic, persistence, security, middleware
API Gateway	Ocelot (ASP.NET Core)	Single entry point, JWT validation, route forwarding, CORS
Messaging	RabbitMQ + MassTransit	Async event-driven communication, consumer retry, DLQ handling
Persistence	SQL Server 2022 + EF Core	Per-service databases via Code-First migrations
Auth	JWT + Google OAuth 2.0 + OTP	Stateless sessions, social login, email-verified signup
Email	MailKit / SMTP	OTP delivery and loan decision notification emails
AI Chatbot	Groq API (Llama-3.1-8b-instant)	Local AI inference, intent detection, voice input via Web Speech API
Charts	Recharts (Angular)	Admin dashboard bar/pie charts for application statistics
Deployment	Docker + Docker Compose	Full containerized local stack, 10 containers, health checks
Testing	NUnit + Moq	Unit tests for service layer across 4 microservices
State Management	Zustand	Auth store with sessionStorage persistence

3.2 System Architecture

CapFinLoan follows a gateway-first microservices pattern inspired by Domain-Driven Design principles. The Angular frontend communicates exclusively through the Ocelot API Gateway, which handles JWT validation and routes each path prefix to the appropriate backend service. Services communicate both synchronously (via HTTP for direct queries) and asynchronously (via

RabbitMQ for event-driven workflows). Each service maintains its own SQL Server database, enforcing strict data ownership boundaries.

The architecture is structured in four distinct layers: the Client/Presentation layer comprising the Angular SPA, the API Gateway layer handling cross-cutting concerns, the Backend Microservices layer containing the seven domain services, and the Shared Infrastructure layer comprising RabbitMQ, SQL Server, and document storage. This layered approach ensures that concerns are cleanly separated, each service can evolve independently, and the system remains resilient to individual component failures.

3.2.1 High-Level Architecture Diagram

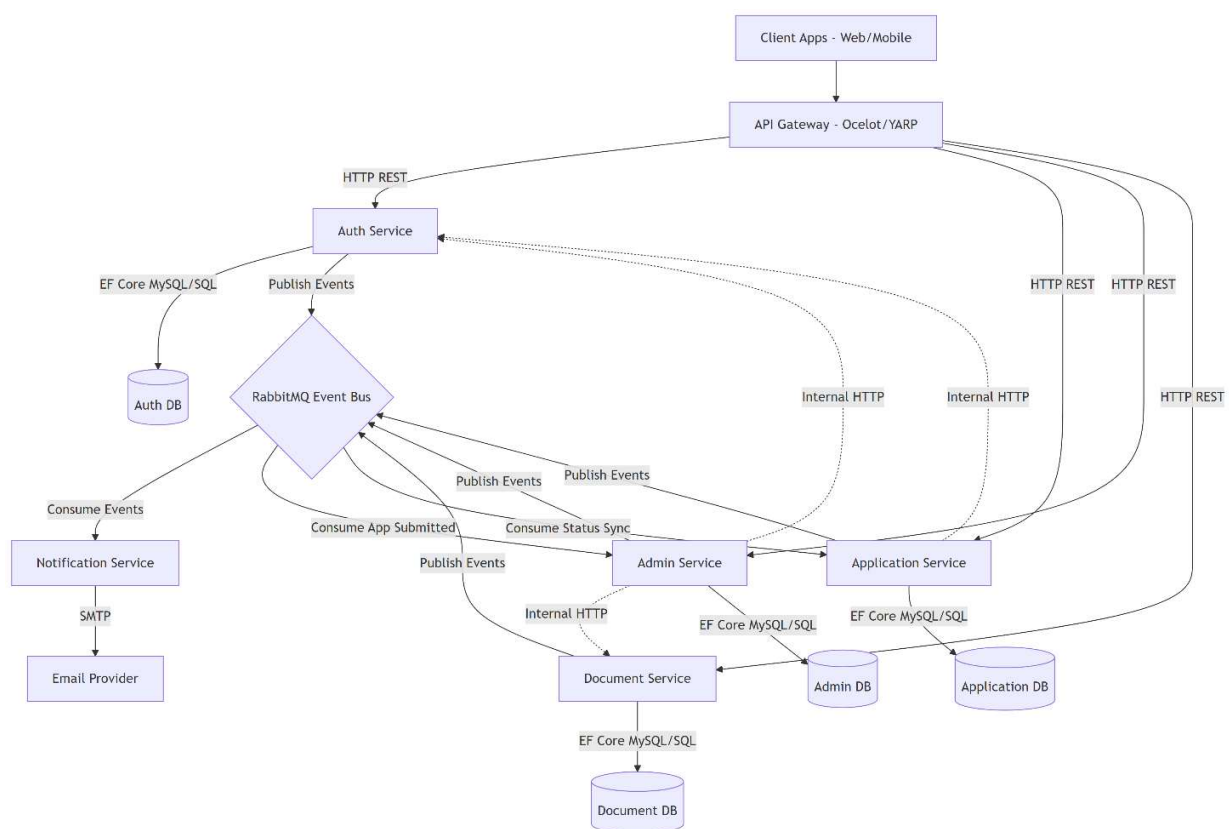


Figure 1: CapFinLoan — “High-Level System Architecture showing all services, RabbitMQ event flows, API Gateway routing, and database ownership

3.2.2 Microservices Overview

The solution includes six backend .NET microservices (Auth, Application, Document, Admin, Notification, Wallet) plus a separate Python ChatbotService. Each service has a well-defined responsibility boundary. The table below provides a concise summary of each service's role, port, and RabbitMQ participation. WalletService runs on port 5027 and is exposed via /gateway/wallet/* through the API Gateway.

Service	Port	Responsibility	RabbitMQ Role
AuthService	5021	JWT auth, OTP signup, Google OAuth, profile management	Publisher (OtpSendEvent)
ApplicationService	5022	Loan application lifecycle, status transitions, submission	Consumer (ApplicationStatusChangedEvent)
DocumentService	5023	Document upload, verification, status sync	Publisher (ApplicationStatusChangedEvent, DocumentVerifiedEvent, ApplicationStatusChangedEvent)
AdminService	5024	Admin decisions, document verification, reports, dashboard	Publisher (LoanApprovedEvent / LoanRejectedEvent, ApplicationStatusChangedEvent); Consumer (DocumentVerifiedEvent)
NotificationService	5095	Email delivery — OTP and loan decisions via SMTP	Consumer (OtpSendEvent, LoanApprovedEvent / LoanRejectedEvent)
ChatbotService	5200	AI chatbot proxy to Groq API, intent detection, app context	None —" stateless proxy
API Gateway	5020	Single entry point, JWT validation, route forwarding (Ocelot)	None —" infrastructure service

3.3 RabbitMQ Messaging Architecture

Asynchronous messaging is a cornerstone of the CapFinLoan architecture. RabbitMQ, configured through the MassTransit abstraction library, enables decoupled communication between services without requiring them to be simultaneously available or to know each other's network addresses directly. This design improves resilience, scalability, and maintainability.

Three primary operational messaging flows govern the system's event-driven behaviour. In the OTP signup flow, AuthService generates a one-time password, hashes it with PBKDF2, stores it with a 10-minute expiry, and publishes an OtpSendEvent containing the plaintext OTP code, applicant name, and email. NotificationService consumes this event and dispatches the OTP email via SMTP. This decoupling means that even if NotificationService is temporarily unavailable, the OTP event will be delivered once it recovers, thanks to RabbitMQ's durable queue guarantees.

In the document status synchronization flow, when a document action (upload, verify, or reject) changes the overall document status for an application, DocumentService publishes an ApplicationStatusChangedEvent. ApplicationService consumes this command and updates its own database with the new loan application status. This maintains consistency across the domain boundary without requiring a shared database or direct HTTP coupling.

In the decision notification flow, after an admin approves or rejects a loan application, AdminService publishes a LoanApprovedEvent / LoanRejectedEvent containing the application ID, applicant details, decision outcome, sanction amount (if approved), interest rate, and decision timestamp. NotificationService consumes this event and sends a decision email to the applicant, ensuring they are promptly informed of the outcome.

RabbitMQ Messaging Contract Summary

Event / Command	Publisher	Consumer	Key Payload Fields
OtpSendEvent	AuthService	NotificationService	Email, OtpCode, ApplicantName, ExpiresAt
ApplicationStatusChangedEvent	DocumentService	ApplicationService	ApplicationId, NewStatus, Remarks, ChangedBy
LoanApprovedEvent / LoanRejectedEvent	AdminService	NotificationService	ApplicationId, ApplicantEmail, Decision, SanctionAmount, InterestRate
ApplicationSubmittedEvent	ApplicationService	DocumentService (trace)	CorrelationId, ApplicationId, OccurredAtUtc, UserId
ApplicationStatusChangedEvent	DocumentService	NotificationService	CorrelationId, ApplicationId, OccurredAtUtc
DocumentVerifiedEvent	DocumentService	AdminService (trace)	CorrelationId, ApplicationId, OccurredAtUtc
ApplicationStatusChangedEvent	AdminService	Saga trace only	CorrelationId, ApplicationId, Remarks

MassTransit Configuration

- Consumers retry: 3 retries with 5-second intervals between attempts.
- After retry exhaustion: fault queue (DLQ) created automatically with warning log entry.
- No HTTP fallback — broker failure throws naturally to preserve consistency.
- DocumentService StatusSyncMode: RabbitMqPrimary — RabbitMQ is the authoritative sync channel.

3.4 Saga Choreography Design

CapFinLoan implements a choreography-based saga pattern to coordinate the loan lifecycle across multiple services. Unlike orchestration (which uses a central coordinator that explicitly directs each step), choreography relies on each service reacting to events published by other

services. This approach avoids the single point of failure and complexity that a central orchestrator would introduce.

The saga begins when an applicant submits a loan application. `ApplicationService` transitions the record from Draft to Submitted status and publishes an `ApplicationSubmittedEvent`. `DocumentService` consumes this event, initializes the document checklist for the application, and publishes a `ApplicationStatusChangedEvent`. `NotificationService` consumes this to send the applicant an email listing the required documents.

Once the applicant uploads documents and `DocumentService` evaluates that all required documents are in a verified state, it publishes a `DocumentVerifiedEvent`. `AdminService` consumes this event in a trace-only capacity — critically, it does NOT automatically transition the application to UnderReview. The system intentionally requires a human-in-the-loop: the admin must manually click 'Mark as Under Review' in the admin review interface. This design decision reflects the real-world regulatory requirement for deliberate human oversight in lending decisions.

When the admin marks the application as UnderReview, `AdminService` publishes an `ApplicationStatusChangedEvent`. When the final approve or reject decision is made, `AdminService` publishes a `LoanApprovedEvent` / `LoanRejectedEvent`, which triggers the applicant notification email through `NotificationService`.

Saga state does not persist in a separate state machine or table. Instead, each service relies on its own domain state (the application status in `ApplicationService`'s database, document statuses in `DocumentService`'s database, and decision records in `AdminService`'s database) as the authoritative source of saga progress. The `CorrelationId` carried in all saga events provides full traceability across service boundaries.

3.5 Authentication and Security Architecture

CapFinLoan implements a multi-layered security architecture designed to protect user data, secure API endpoints, and prevent unauthorized access. The system supports three distinct authentication pathways, each appropriate for different user scenarios.

3.5.1 OTP-Based Email Signup

New applicants register through a two-step process. In the first step, the applicant provides their email address and receives a one-time password via email (delivered asynchronously through `RabbitMQ` and `NotificationService`). The OTP is hashed using PBKDF2 HMACSHA256 with a 10-minute expiry window and stored in the `AuthService` database. In the second step, the applicant submits the OTP along with their registration details. `AuthService` validates the OTP hash, creates the user account, and returns a JWT token.

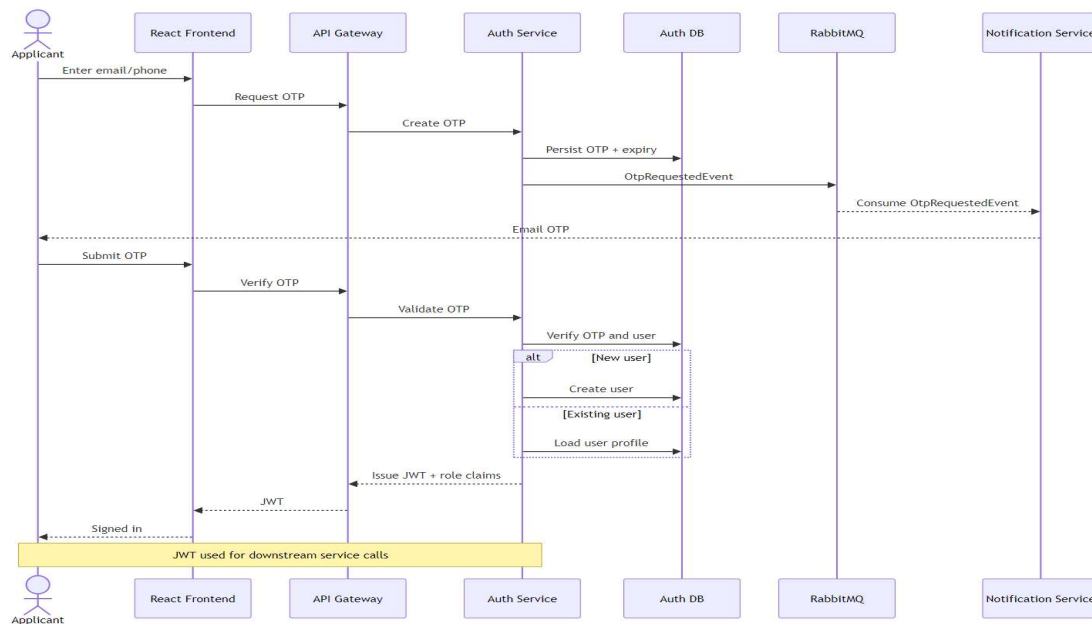


Figure 2: OTP-Based Signup Sequence Diagram — showing the two-step flow from OTP request through RabbitMQ to email delivery and account creation

3.5.2 Google OAuth 2.0

The Google OAuth flow is handled entirely server-side for security. The Angular frontend obtains a Google ID token using the Google Sign-In (GSI) JavaScript library and posts it to AuthService. AuthService validates the token by calling the Google token info endpoint and verifying that the 'aud' claim matches the configured Google Client ID. For new users, an account is created with a null PasswordHash (no password is set). For existing Google users, a fresh JWT is issued. If an existing local (email/password) account shares the same email, a 400-conflict response is returned.

3.5.3 JWT and API Gateway Security

All protected API routes require a valid JWT Bearer token, validated at the Ocelot API Gateway level. The JWT contains the user's email and role claims, signed with HMACSHA256. The gateway performs token signature validation, and expiry checks before forwarding any request to a downstream service. Admin-specific routes require the ROLE_ADMIN claim in addition to a valid token. The Zustand auth store in the Angular frontend persists the token in sessionStorage and the Axios interceptor automatically attaches the Authorization header to every outgoing request.

Security Concern	Implementation	Production Recommendation
Password Hashing	ASP.NET Core IPasswordHasher (PBKDF2 HMACSHA256)	Add password policy enforcement and breached-password checks

Security Concern	Implementation	Production Recommendation
JWT	HMACSHA256-signed, validated at Ocelot gateway	Add refresh-token rotation and shorter access-token lifetime
Google OAuth	tokeninfo endpoint + aud claim validation	Use Google JWT libraries for offline validation
OTP	PBKDF2 hashed, 10-minute expiry, marked as used on consumption	Rate-limit OTP requests per email address
Secrets	.env file (gitignored), Docker env vars	Use a secret manager (Azure Key Vault / AWS Secrets Manager)
Document Access	JWT required; blob served with auth header check	Add document-level ownership validation
Admin Routes	ADMIN role claim at gateway + controller authorization	Add method-level authorization review for all endpoints
CORS	localhost:5173 and localhost:3000 only	Restrict to production domain origins
SQL Injection	EF Core parameterized queries throughout	Periodic security scan with static analysis tools

3.6 Loan Application Lifecycle

The loan application lifecycle is the central business process of the CapFinLoan platform. It governs how a loan request progresses from initial creation through administrative review to a final credit decision. The lifecycle is implemented across three services — ApplicationService (owns the application record), DocumentService (owns document state), and AdminService (owns decisions and status history) — with coordination achieved through RabbitMQ messaging.

The lifecycle begins when an authenticated applicant accesses the 'Apply Loan' page, which presents a four-step wizard collecting Personal Information, Employment Details, Loan Details, and a final Review screen. The applicant can save a draft at any point without submitting, allowing them to return and edit the application. Once satisfied, the applicant submits the application with a declaration acceptance.

After submission, the application enters the DocsPending state following the first document activity (not immediately on submission). The applicant is expected to upload the four required documents: ID Proof, Address Proof, Bank Statement, and Income Proof. Each document upload is recorded in DocumentService's database with its own status (Pending, Verified, Rejected).

When an administrator reviews the application and verifies all documents, the application transitions to DocsVerified. The admin can then manually mark it as UnderReview to indicate active examination, and subsequently issue a final Approve or Reject decision. Approved decisions include a sanction amount and interest rate. Both decisions trigger an email notification to the applicant.

Status Lifecycle Overview

Status	Triggered By	Next Possible States
Draft	Application creation / save draft	Submitted
Submitted	Applicant submits application	DocsPending (on first document activity)
DocsPending	First document upload or action	DocsVerified (all docs verified)
DocsVerified	All documents verified by admin	UnderReview
UnderReview	Admin manually marks as Under Review	Approved, Rejected
Approved	Admin approves with sanction details	Final — no further transitions
Rejected	Admin rejects with remarks	Final — no further transitions

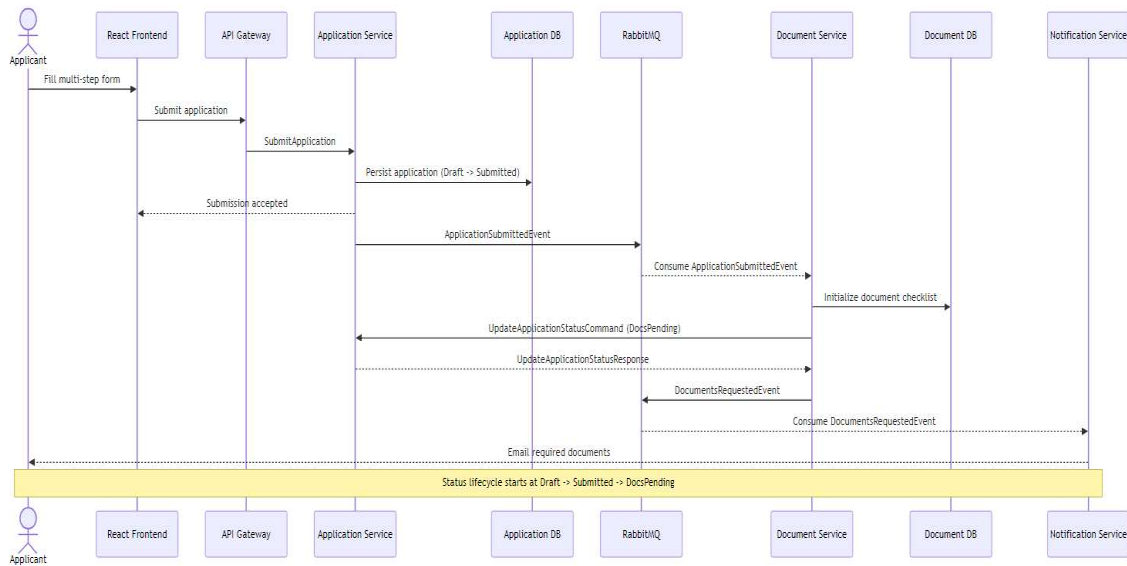


Figure 3: Application Submission Sequence Diagram — Draft — Submitted — DocsPending lifecycle across ApplicationService, DocumentService, RabbitMQ, and NotificationService

3.7 API Gateway and Routes

The Ocelot API Gateway (port 5020) serves as the single-entry point for all frontend API calls. It handles JWT validation, CORS configuration, and request routing to the appropriate downstream service based on path prefix matching. This design shields the frontend from knowledge of individual service addresses and enables centralized security enforcement. Active routes include `/gateway/auth/*` → `authservice:5021`, `/gateway/applications/*` → `applicationservice:5022`, `/gateway/documents/*` → `documentservice:5023`, `/gateway/admin/*` → `adminservice:5024`, `/gateway/wallet/*` → `walletservice:5027`, and `/gateway/chat` → `chatbotservice:5026`.

Method	Gateway Route	Downstream Service	Auth Required
POST	<code>/gateway/auth/login</code>	<code>authservice:5021</code>	None
POST	<code>/gateway/auth/signup</code>	<code>authservice:5021</code>	None
POST	<code>/gateway/auth/signup/send-otp</code>	<code>authservice:5021</code>	None
POST	<code>/gateway/auth/google</code>	<code>authservice:5021</code>	None
GET/PUT	<code>/gateway/auth/profile</code>	<code>authservice:5021</code>	JWT
PUT	<code>/gateway/auth/profile/password</code>	<code>authservice:5021</code>	JWT
GET/POST/PUT/DELETE	<code>/gateway/applications/*</code>	<code>applicationservice:5022</code>	JWT
GET/POST/PUT	<code>/gateway/documents/*</code>	<code>documentservice:5023</code>	JWT
GET/POST/PUT	<code>/gateway/admin/*</code>	<code>adminservice:5024</code>	JWT + ADMIN role

Method	Gateway Route	Downstream Service	Auth Required
POST	/gateway/chat	chatbotservice:5026	None

3.8 Frontend Architecture

The Angular 18 frontend is a single-page application built with Vite as the build tool and Tailwind CSS for utility-first styling. The application follows a page-based component architecture with a clean separation between applicant-facing pages, admin-facing pages, shared components, and service layer modules.

Route protection is implemented using two custom route guards: `RequireAuth` (which checks for a valid JWT token in the Zustand auth store) and `RequireAdmin` (which additionally validates the `ROLE_ADMIN` claim). The Axios instance is configured with a base URL pointing to the API Gateway and an interceptor that automatically attaches the `Authorization: Bearer` header to every outgoing request. A response interceptor handles 401 Unauthorized responses by clearing the auth store and redirecting to the login page.

3.8.1 Applicant Pages

The applicant-side interface consists of five primary pages accessible after authentication. The Dashboard page displays KPI cards summarizing total applications, pending/active applications, and approved applications, along with a table of the most recent five applications. The My Applications page shows all applications for the logged-in user with client-side search by name, status filter, and pagination at ten records per page.

The Apply Loan wizard is the most complex page, featuring a four-step form. Step 1 collects personal details (name, date of birth, gender, address, contact information). Step 2 captures employment details (employer name, employment type, monthly income, annual income). Step 3 records loan details (requested amount, tenure in months, loan purpose, remarks). Step 4 presents a full review before final submission with declaration acceptance.

The Application Detail page aggregates data from three parallel API calls — the application record, status history timeline, and documents — and renders a vertical status timeline alongside four document upload boxes. Document upload is enabled only when the application is in Submitted or DocsPending status. Documents are viewed through blob URLs generated from authenticated fetch requests.

3.8.2 Admin Pages

The admin interface provides four specialized pages. The Admin Dashboard displays six KPI cards reflecting the distribution of applications across all statuses, populated from the admin dashboard endpoint. The Applications Queue page shows a searchable, filterable, paginated list of all applications in the system. The Admin Review page is the core workflow interface, presenting full application details on the left, document boxes with verify/reject controls, and a decision panel on the right. The Admin Reports page visualizes application statistics using Recharts bar and pie charts and supports CSV export.

3.9 AI Chatbot Integration

A key differentiating feature of CapFinLoan is its integrated AI chatbot assistant, accessible to all authenticated applicants through a floating widget rendered in the bottom-right corner of every applicant page. The chatbot is powered by a locally hosted Groq API instance running the llama-3.1-8b-instant language model, with inference handled by ChatbotService acting as a proxy.

ChatbotService implements intent detection logic that classifies incoming messages before forwarding them to the language model. When an applicant asks about their latest application, all applications, applications by status, or a specific application by ID, ChatbotService fetches the relevant data from ApplicationService and includes it in the context passed to the language model. This allows the AI to provide accurate, personalized responses grounded in actual application data rather than generic information.

Intent	Trigger Keywords	Data Fetched from ApplicationService
LATEST_APPLICATION	latest, recent, newest	GET /api/applications/my — first result only
APPLICATIONS_BY_STATUS	approved, rejected, pending, docs_pending, etc.	GET /api/applications/my — filtered by status
ALL_APPLICATIONS	all my applications, list applications, show all	GET /api/applications/my — top 5 results
SPECIFIC_APPLICATION	8+ character hex string in message (application ID)	GET /api/applications/{id} by specific ID
GENERAL	Any message not matching above patterns	None — pure language model response

The chatbot widget features a pulsing ring animation to attract attention until the applicant opens it for the first time. Voice input is supported through the Web Speech API in continuous mode with interim results displayed in the input field as the user speaks. Application result cards rendered by the chatbot are clickable and navigate directly to the application detail page.

3.10 Use Case Diagram

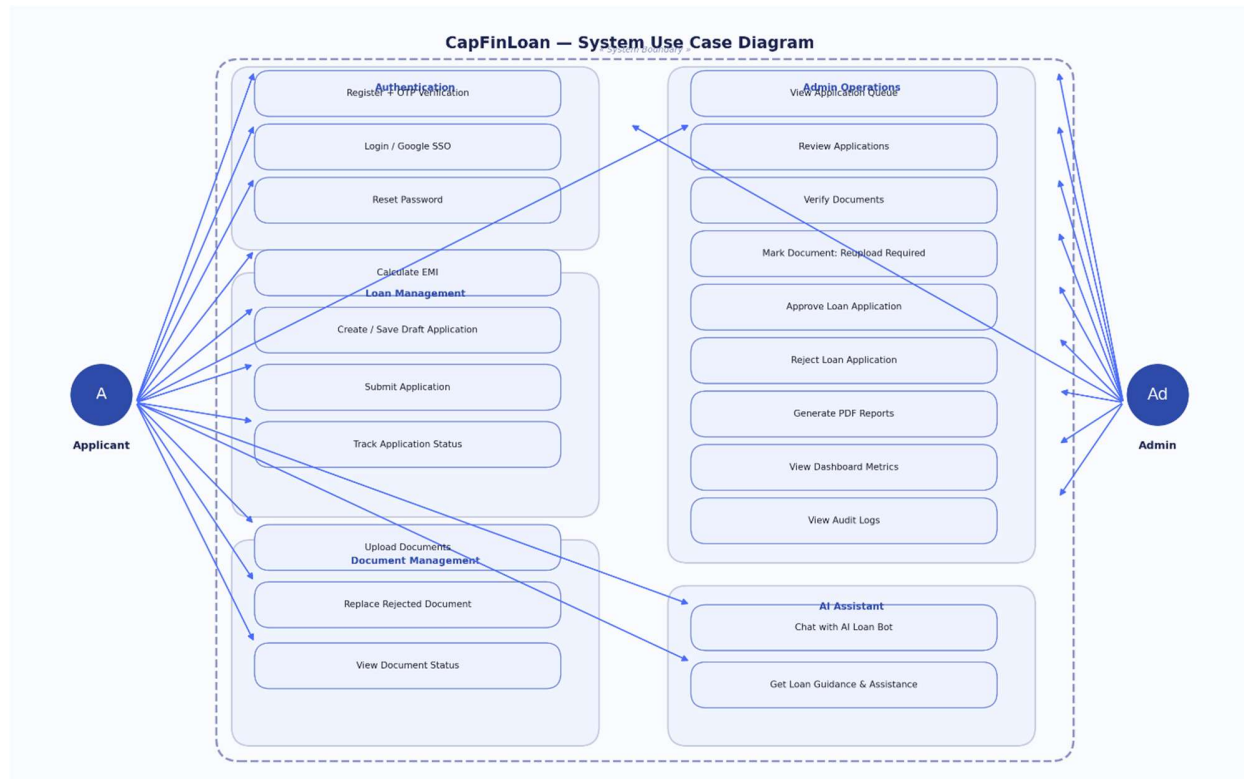


Figure 4: CapFinLoan Use Case Diagram — showing all actor interactions across Access & Identity, Loan Application, Document management, and Admin Processing subsystems

3.11 Database Design

CapFinLoan strictly adheres to the database-per-service microservices principle. Each service that requires persistence owns its own SQL Server database, with no cross-service direct database access. Services communicate about shared domain entities exclusively through API calls and message events. This approach ensures that each service's data model can evolve independently without coordination, and that a schema change in one service cannot break another.

3.11.1 Database Overview

Service	Database Name	Key Tables
AuthService	CapFinLoanAuthDb	auth.Users, auth.SignupOtps
ApplicationService	CapFinLoanAppDb	core.LoanApplications, core.StatusHistory
DocumentService	CapFinLoanDocsDb	docs.Documents
AdminService	CapFinLoanAdminDb	admin.Decisions, admin.StatusHistory

Service	Database Name	Key Tables
NotificationService	None (stateless)	No database — pure event consumer
ChatbotService	None (stateless)	No database — pure proxy service

3.11.2 Entity-Relationship Diagram

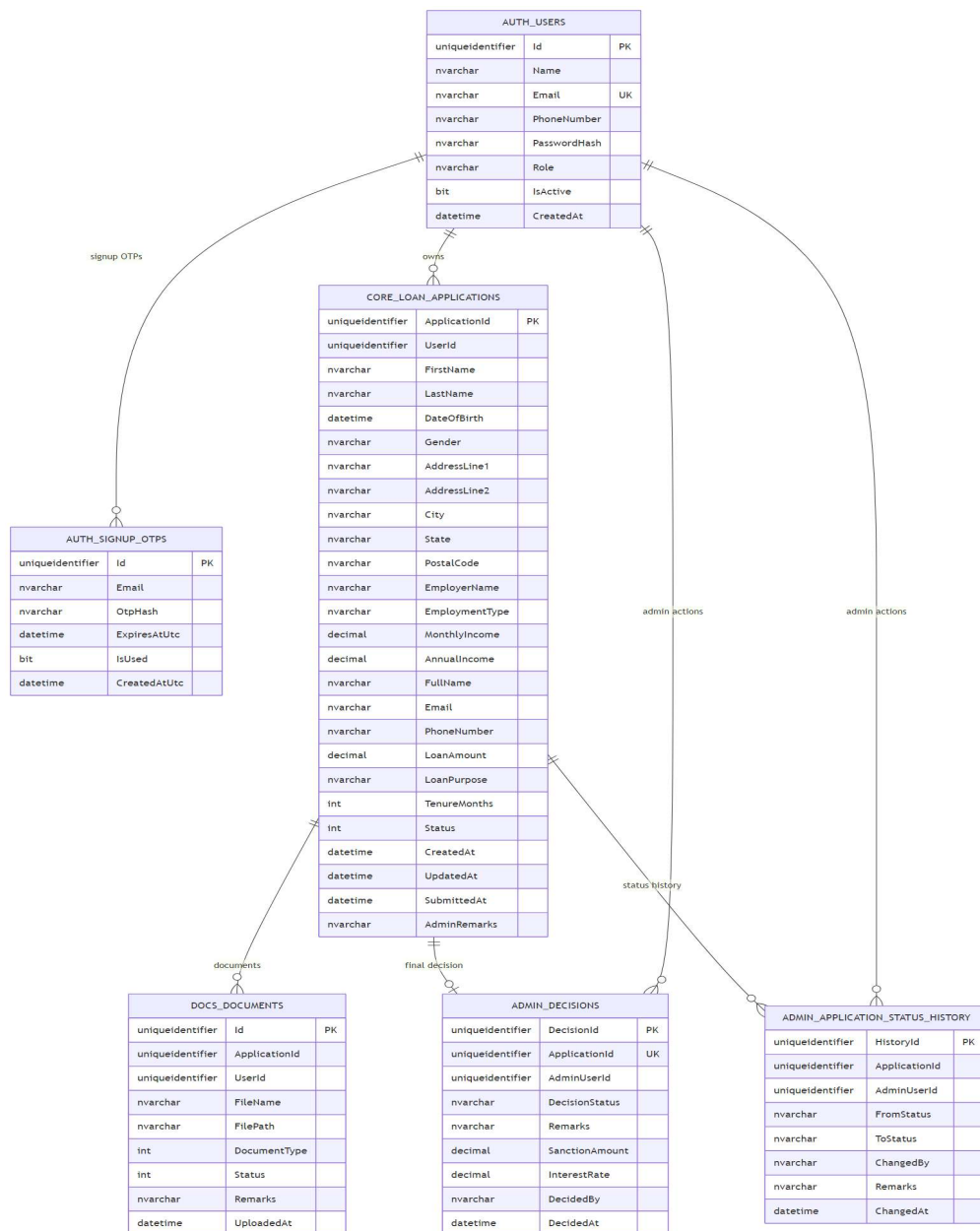


Figure 5: CapFinLoan Database Entity-Relationship Diagram — cross-service entity relationships showing AUTH_USERS, CORE_LOAN_APPLICATIONS, DOCS_DOCUMENTS, ADMIN_DECISIONS, and ADMIN_APPLICATION_STATUS_HISTORY

3.11.3 Key Schema Details

The auth.Users table stores all user accounts with support for both local (email/password) and Google OAuth users. The PasswordHash field is nullable to accommodate Google users who have no local password. The AuthProvider field distinguishes between LOCAL and GOOGLE accounts, and the GoogleId field stores the Google 'sub' claim for OAuth users.

Column	Type	Notes
Id	GUID (PK)	Unique user identifier
Name	nvarchar	Full name
Email	nvarchar (Unique)	Email address — unique constraint
PasswordHash	nvarchar (Nullable)	PBKDF2 hash — null for Google OAuth users
Role	nvarchar	APPLICANT or ADMIN
IsActive	bit	Account active/inactive flag
PhoneNumber	nvarchar (Nullable)	Optional contact number
AuthProvider	nvarchar	LOCAL or GOOGLE
GoogleId	nvarchar (Nullable)	Google sub claim — null for local users
CreatedAt	datetime2	Account creation timestamp

The admin.Decisions table records the final credit decision for each application. The ApplicationId has a unique constraint ensuring only one binding decision can exist per application. The SanctionAmount and InterestRate are set to zero for rejected applications. The DecidedBy field stores the admin's email for audit purposes.

Column	Type	Notes
DecisionId	GUID (PK)	Unique decision identifier
ApplicationId	GUID (Unique)	References the loan application — one decision per application
Decision	nvarchar	APPROVED, REJECTED, or UNDER_REVIEW
Remarks	nvarchar	Admin remarks for the decision
SanctionAmount	decimal	Zero if rejected; approved amount if approved
InterestRate	decimal	Zero if rejected; configured rate if approved
DecidedAt	datetime2	Timestamp of the decision
DecidedBy	nvarchar	Email of the admin who made the decision

3.12 Clean Architecture

Each microservice in CapFinLoan is structured according to Clean Architecture principles, ensuring a clear separation of concerns and unidirectional dependency flow. This architecture makes each service independently testable, maintainable, and extensible without changes propagating unexpectedly across layers.

Layer	Contents	Dependencies
Domain Layer	Entities, Enums, Business rules, Domain exceptions	None — no external dependencies
Application Layer	Service interfaces, DTOs, business logic implementations, exception handling middleware	Depends on Domain only
Infrastructure Layer	EF Core DbContext, JWT utilities, PasswordHasher, MassTransit consumers/publishers, HTTP clients	Depends on Application and Domain
API / Presentation Layer	Controllers, Program.cs, Dockerfiles, configuration files	Depends on Application and Infrastructure

Dependencies point strictly inward — the outer layers depend on inner layers, never the reverse. This ensures that domain logic and business rules remain isolated from framework-specific implementation details, making the core business logic portable and unit-testable without requiring a running database, message broker, or web server.

3.13 Unit Testing Strategy

A comprehensive unit testing suite is implemented across four of the seven microservices. Tests use the NUnit testing framework and the Moq library for mocking dependencies. All tests target the service layer and operate exclusively in memory — no real database connections, RabbitMQ brokers, or SMTP servers are required. This approach ensures tests run fast, reliably, and in isolation.

Test Project	Service Under Test	Key Test Cases
CapFinLoan.Auth.Tests	AuthService.cs	Login valid/invalid/inactive user; OTP signup valid/expired/wrong code/duplicate email; Google OAuth new user/existing Google user/existing LOCAL user conflict
CapFinLoan.Application.Tests	LoanApplicationService.cs	Submit from Draft (success); submit from non-Draft status (failure); non-existent application; valid status transition; invalid transition enforcement; GetMyApplications pagination

Test Project	Service Under Test	Key Test Cases
CapFinLoan.Admin.Tests	AdminService.cs	Approve publishes LoanApprovedEvent / LoanRejectedEvent; reject publishes LoanApprovedEvent / LoanRejectedEvent; double-decision attempt fails with conflict; dashboard KPI counts are computed correctly
CapFinLoan.Document.Tests	DocumentService.cs	All documents verified publishes DocsVerifiedEvent; partial verification DocsPending; any rejection DocsPending; verify document updates status; reject document updates status with remarks

3.14 Docker and Deployment

The entire CapFinLoan platform is containerized using Docker and orchestrated through a single Docker Compose configuration file. All containers share a common Docker network, enabling inter-service communication using container names as hostnames. The Ocelot gateway's ocelot.json uses container names (e.g., authservice:5021) rather than localhost addresses, ensuring proper network resolution within the Docker environment.

3.14.1 Container Architecture

Container	Type	Port(s)	Depends On
sqlserver	SQL Server 2022	1433	Nothing
rabbitmq	RabbitMQ 3 + Management UI	5672, 15672	Nothing
authservice	ASP.NET Core 8	5021	sqlserver (healthy), rabbitmq (healthy)
applicationservice	ASP.NET Core 8	5022	sqlserver (healthy), rabbitmq (healthy)
documentservice	ASP.NET Core 8	5023	sqlserver (healthy), rabbitmq (healthy)
adminservice	ASP.NET Core 8	5024	sqlserver (healthy), rabbitmq (healthy)
notificationsservice	ASP.NET Core 8	5095	rabbitmq (healthy)
chatbotsservice	ASP.NET Core 8	5200	applicationservice
gateway	ASP.NET Core 8 / Ocelot	5020	All backend services
frontend	Angular 18 (Nginx)	3000 : 80	gateway

3.14.2 Environment Configuration

The system uses a .env file (gitignored) for all sensitive configuration values, injected into containers as environment variables via Docker Compose. ASP.NET Core's double-underscore notation is used to override nested configuration sections from environment variables:

Variable	Used By	Purpose
SA_PASSWORD	SQL Server, all backend services	SQL Server SA password and service connection strings
EMAIL_SENDER	NotificationService	Gmail address for outgoing SMTP email
EMAIL_APP_PASSWORD	NotificationService	Gmail App Password for SMTP authentication
GOOGLE_CLIENT_ID	AuthService	Google OAuth Client ID for server-side token validation
VITE_GOOGLE_CLIENT_ID	Angular Frontend	Google Client ID for GSI library initialization
JWT_SECRET	AuthService, API Gateway	HMACSHA256 secret for JWT signing and validation

3.14.3 Development URLs

Service	Local URL
Frontend (local dev)	http://localhost:5173
Frontend (Docker/Nginx)	http://localhost:3000
API Gateway	http://localhost:5020
RabbitMQ Management UI	http://localhost:15672 (guest/guest)
AuthService Swagger	http://localhost:5021/swagger
ApplicationService Swagger	http://localhost:5022/swagger
DocumentService Swagger	http://localhost:5023/swagger
AdminService Swagger	http://localhost:5024/swagger
Groq API AI Endpoint	http://localhost:1234

3.15 Design Patterns Used

CapFinLoan applies numerous well-established software design patterns at various levels of the system, from high-level architectural patterns down to individual service design decisions:

Pattern	Applied In	Description
Gateway Pattern	Ocelot API Gateway (port 5020)	Single backend entry point handling JWT validation, routing, and CORS
Database-per-Service	All 4 persistent services	Each service owns its isolated SQL Server database — no cross-service DB access
Choreography Saga	Cross-service loan lifecycle	Event-driven coordination without central orchestrator; human-in-the-loop for admin review
Publish-Subscribe	RabbitMQ + MassTransit	Publishers emit events; subscribers angular independently; decoupled event-driven communication
Repository Pattern	All services (EF Core)	Data access abstracted through EF Core repositories; service layer remains DB-agnostic
DTO Pattern	All services	Request/response objects decouple API contracts from internal domain entity models
Proxy Pattern	ChatbotService	Acts as a proxy to Groq API, adding intent detection and application context enrichment
Strategy Pattern (implicit)	AuthService	Multiple authentication strategies (local, Google OAuth, OTP) behind a common interface
Retry Pattern	MassTransit consumer config	3-retry policy with 5-second intervals before DLQ escalation
Clean Architecture	All services	Layered inward-only dependencies: Domain — Application. Infrastructure API

3.16 Reliability and Fault Tolerance

CapFinLoan incorporates several reliability mechanisms to ensure graceful handling of failures at different system layers. The MassTransit consumer retry policy automatically retries failed message processing up to three times before moving the message to a dead-letter queue. This handles transient failures such as temporary database unavailability or network timeouts without message loss.

Docker Compose health checks ensure that dependent services (SQL Server, RabbitMQ) are fully healthy before backend services attempt to connect. Services that consume from RabbitMQ are configured to wait for the broker's health check to pass, preventing startup failures due to race conditions. The API Gateway's Ocelot configuration uses container names for downstream service addressing, enabling Docker's internal DNS to resolve service locations dynamically.

For document upload failures, the DocumentService maintains a per-document status that allows partial retries — a failed upload for one document type does not affect the status of other documents. The Admin Review page disables decision buttons once an application has reached UNDER_REVIEW, APPROVED, or REJECTED status, preventing accidental duplicate decisions at the UI layer as well as the service layer (which enforces a unique constraint on ApplicationId in the decisions table).

Failure Scenario	System Behavior
RabbitMQ temporarily unavailable	Services fail to start gracefully; Docker health check delays startup; no message loss once broker recovers
Message consumer processing failure	MassTransit retries 3 times with 5-second delay; on exhaustion, message moves to DLQ with warning log
Duplicate admin decision attempt	AdminService returns conflict error; decisions table unique constraint on ApplicationId prevents duplicates at DB level
Groq API AI model unreachable	ChatbotService returns a graceful fallback message; no exception thrown; applicant sees a friendly error message
Document upload while in wrong status	Frontend disables upload UI; backend validates status before accepting upload; consistent double protection

Chapter 4: Conclusion

CapFinLoan represents a comprehensive, industry-aligned full-stack application developed during the internship at Capgemini Technology Services India Limited. The project successfully demonstrates the complete digital lending journey — from user registration and multi-step loan application creation through document management, administrative review, final credit decision, and asynchronous email notification — across a well-structured microservices system built on modern, production-relevant technologies.

From an architectural standpoint, the project showcases several advanced software engineering concepts that are directly applicable to real-world enterprise systems. The clean separation of seven independently deployable services, each with its own database, ensures that the system can scale and evolve without unintended cross-service coupling. The choreography-based saga pattern demonstrates a mature approach to distributed transaction coordination that avoids the brittleness of two-phase commit while preserving business logic integrity. The deliberate human-in-the-loop design for administrative review reflects an understanding of regulatory requirements in financial services.

The technology breadth demonstrated across the project is significant. On the backend, ASP.NET Core 8 with EF Core, MassTransit/RabbitMQ messaging, Ocelot gateway, and Clean Architecture provide a robust and maintainable service foundation. On the frontend, Angular 18 with Zustand state management, recharts data visualization, and the Web Speech API for voice input demonstrate proficiency across the full client-side stack. The AI chatbot integration using a locally hosted LLM adds a modern intelligent interaction layer that enriches the user experience.

The internship has provided hands-on experience in microservices architecture, event-driven systems design, cloud-native deployment patterns, secure authentication flows, and modern DevOps practices. The skills developed — particularly in distributed systems communication patterns, database isolation strategies, and containerization — are directly transferable to production software engineering roles.

The project is suitable as a portfolio-grade submission demonstrating readiness for professional software engineering responsibilities, and the production improvement roadmap outlined provides a clear path toward further enterprise-grade hardening through Kubernetes deployment, circuit breaker patterns, managed database services, and enhanced observability tooling.

This internship experience at Capgemini, combined with the technical foundation built through the academic program at Lovely Professional University, has been immensely valuable in bridging the

gap between theoretical knowledge and practical engineering skills required in today's competitive software industry.

— Yelugotla Sanket | Registration No: 12223854

B.Tech — Computer Science and Engineering | Lovely Professional University

Chapter 5: References

Books and Academic References

- [1] A. Lock and S. Crawley, ASP.NET Core in Action, 3rd ed. Manning Publications, 2023.
- [2] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. O'Reilly Media, 2021.
- [3] C. Richardson, Microservices Patterns: With Examples in Java. Manning Publications, 2018.
- [4] A. Freeman, Pro ASP.NET Core 8. Apress, 2024.
- [5] R. Wieruch, The Road to Angular. Self-published, 2023.

Online Documentation and Resources

- [6] Microsoft ASP.NET Core Documentation. <https://learn.microsoft.com/en-us/aspnet/core/>
- [7] Mass Transit Documentation. <https://masstransit.io/documentation>
- [8] Ocelot API Gateway Documentation. <https://ocelot.readthedocs.io/>
- [9] Angular Official Documentation. <https://angular.dev/>
- [10] Tailwind CSS Documentation. <https://tailwindcss.com/docs/>
- [11] RabbitMQ Documentation. <https://www.rabbitmq.com/documentation.html>
- [12] Docker Documentation. <https://docs.docker.com/>
- [13] Entity Framework Core Documentation. <https://learn.microsoft.com/en-us/ef/core/>
- [14] Groq API Documentation. <https://console.groq.com/docs>
- [15] Capgemini Technology Services India Limited — Internal Project Guidelines and Training Materials, 2025 — 2026.

Internship at: Capgemini Technology Services India Limited (17 Dec 2025 — 20 May 2026)

Designation: Analyst / A4 (Trainee) | Superset ID: 6872886